Autonomous Agents · Follow publication

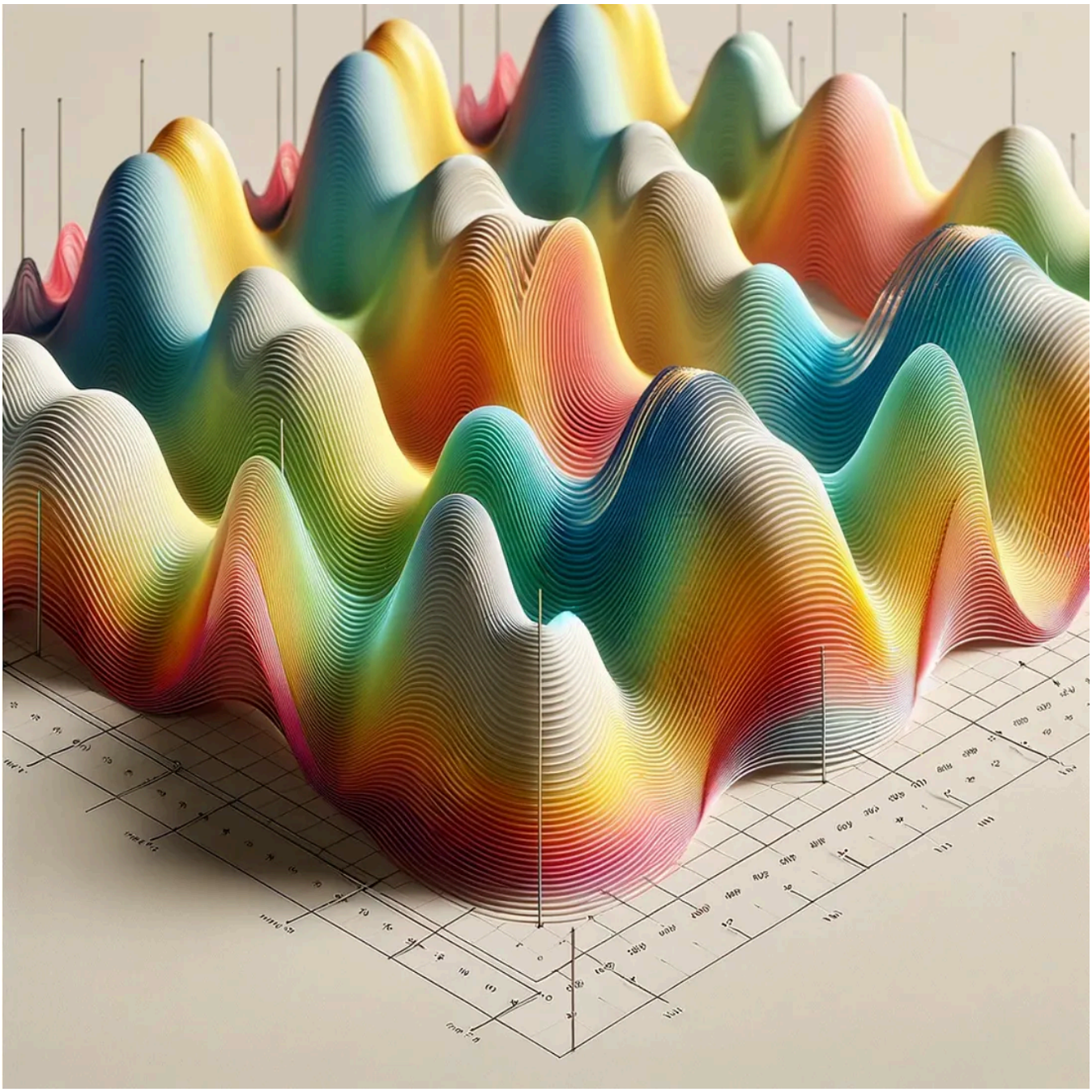
Math Behind Positional Embeddings in Transformer Models

6 min read · May 29, 2024

Freedom Preetham Follow

Listen Share

Positional embeddings are a fundamental component in transformer models, providing critical positional information to the model. This blog will explore the mathematical concepts behind positional embeddings, particularly focusing on sinusoidal positional embeddings, and will work through a detailed example to illustrate these concepts.



Transformer models, unlike recurrent neural networks (RNNs), process tokens in a sequence in parallel. This parallelism means that transformers lack inherent knowledge of the order of tokens. To address this, positional embeddings are introduced. These embeddings encode the position of each token in the sequence, enabling the model to understand the order.

Mathematical Concept of Positional Embeddings

Positional embeddings are added to the input token embeddings to provide the model with information about the positions of the tokens. We will focus on sinusoidal positional embeddings, as introduced by Vaswani et al. in the original Transformer paper.

$$PE_{(\text{pos},\ 2i)} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(\text{pos},\ 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

Where:

- pos is the position of the token in the sequence.
- i is the specific dimension within the embedding vector.
- d is the dimensionality of the embeddings.
- 10000^2i/d ensures that each dimension of the positional embedding has a different frequency.

Why 10000? Frequency and Wavelength ~

The choice of 10000 in the denominator is linked to the management of frequencies across the dimensions of the embeddings:

- **Frequency Control:** The frequency of the sinusoidal function is inversely proportional to the argument in the sine or cosine functions. As i increases, 10000^2i/d grows exponentially, decreasing the frequency of the sinusoidal function, which means the wavelength (the distance over which the function repeats itself) increases.
- **Frequency Spectrum:** By using 10000 as the base, the frequency spectrum is spread over a logarithmic scale. This ensures that the embeddings can capture patterns that occur over different sequence lengths. Lower dimensions (small i)

have higher frequencies, capturing fine-grained positional patterns, while higher dimensions (large i) have lower frequencies, capturing broader patterns.

Derivatives and Gradient Flow

The derivatives of the sine and cosine functions are crucial for understanding how these embeddings contribute to the learning process:

- **Derivatives:** The derivative of sin(x) is cos(x), and the derivative of cos(x) is −sin(x). These derivatives are crucial during backpropagation in training as they determine how gradients are passed through the network. A well-chosen frequency ensures that these derivatives do not vanish (leading to vanishing gradients) or become too large (leading to exploding gradients).
- **Stable Gradients:** By setting the denominator to 10000^2i/d, the rate of change of the sinusoidal functions is moderated, aiding stable gradient propagation across a range of positions and dimensions. This stability is vital for effective training, especially in deep neural networks like Transformers.

Generalization to Longer Sequences

One of the significant advantages of sinusoidal positional embeddings is their ability to generalize to sequence positions beyond those seen during training:

- **Extrapolation:** The sinusoidal nature allows the model to infer positional relationships even for sequence lengths not encountered during training, thanks to the predictable, repeating nature of sine and cosine functions.
- **Handling Large Sequences:** The exponential scaling with 10000^2i/d allows embeddings to handle very large positions effectively without requiring retraining or modification to the embedding layer, unlike learned embeddings which are fixed to the maximum sequence length seen during training.

Step-by-Step Example

Let's consider a sequence of three tokens: ["The", "cat", "sat"], with an embedding dimension (d) of 4.

Step 1: Token Embeddings

Assume we have precomputed token embeddings for our words "The", "cat", and "sat". These embeddings are vectors of dimension 4.

- Embedding for "The" (E_The): [0.1, 0.2, 0.3, 0.4]

- Embedding for “cat” (E_cat): [0.5, 0.6, 0.7, 0.8]
- Embedding for “sat” (E_sat): [0.9, 1.0, 1.1, 1.2]

Step 2: Compute Positional Embeddings

Using the sinusoidal positional embedding formula, we compute the positional embeddings for each position in the sequence.

For position 0:

$$PE_{(0,\,0)} = \sin\left(\frac{0}{10000^{0/4}}\right) = \sin(0) = 0$$

$$PE_{(0,\,1)} = \cos\left(\frac{0}{10000^{0/4}}\right) = \cos(0) = 1$$

$$PE_{(0,\,2)} = \sin\left(\frac{0}{10000^{2/4}}\right) = \sin(0) = 0$$

$$PE_{(0,\,3)} = \cos\left(\frac{0}{10000^{2/4}}\right) = \cos(0) = 1$$

So, the positional embedding for position 0 is: [0, 1, 0, 1].

For position 1:

$$PE_{(1,\,0)} = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1) \approx 0.8415$$

$$PE_{(1,\,1)} = \cos\left(\frac{1}{10000^{0/4}}\right) = \cos(1) \approx 0.5403$$

$$PE_{(1,\,2)} = \sin\left(\frac{1}{10000^{2/4}}\right) \approx \sin(0.01) \approx 0.01$$

$$PE_{(1,\,3)} = \cos\left(\frac{1}{10000^{2/4}}\right) \approx \cos(0.01) \approx 0.99995$$

So, the positional embedding for position 1 is: [0.8415, 0.5403, 0.01, 0.99995].

For position 2:

$$PE_{(2,\,0)} = \sin\left(\frac{2}{10000^{0/4}}\right) = \sin(2) \approx 0.9093$$

$$PE_{(2,\,1)} = \cos\left(\frac{2}{10000^{0/4}}\right) = \cos(2) \approx -0.4161$$

$$PE_{(2,\,2)} = \sin\left(\frac{2}{10000^{2/4}}\right) \approx \sin(0.02) \approx 0.02$$

$$PE_{(2,\,3)} = \cos\left(\frac{2}{10000^{2/4}}\right) \approx \cos(0.02) \approx 0.9998$$

So, the positional embedding for position 2 is: [0.9093, −0.4161, 0.02, 0.9998].

Step 3: Combine Token and Positional Embeddings

Now, we add the positional embeddings to the token embeddings element-wise.

For “The” at position 0:

$$H_{\text{The}} = E_{\text{The}} + PE_{(0)}$$

$$H_{\text{The}} = [0.1, 0.2, 0.3, 0.4] + [0, 1, 0, 1]$$

$$H_{\text{The}} = [0.1, 1.2, 0.3, 1.4]$$

For “cat” at position 1:

$$H_{\text{cat}} = E_{\text{cat}} + PE_{(1)}$$

$$H_{\text{cat}} = [0.5, 0.6, 0.7, 0.8] + [0.8415, 0.5403, 0.01, 0.99995]$$

$$H_{\text{cat}} = [1.3415, 1.1403, 0.71, 1.79995]$$

For “sat” at position 2:

$$H_{\text{sat}} = E_{\text{sat}} + PE_{(2)}$$

$$H_{\text{sat}} = [0.9, 1.0, 1.1, 1.2] + [0.9093, -0.4161, 0.02, 0.9998]$$

$$H_{\text{sat}} = [1.8093, 0.5839, 1.12, 2.1998]$$

Final Embeddings

- Embedding for “The”: [0.1, 1.2, 0.3, 1.4]
- Embedding for “cat”: [1.3415, 1.1403, 0.71, 1.79995]
- Embedding for “sat”: [1.8093, 0.5839, 1.12, 2.1998]

These combined embeddings now contain both the semantic information from the token embeddings and the positional information from the positional embeddings.

Strengths and Weakness of Positional Encodings

Recap: Sinusoidal Positional Embeddings

Sinusoidal positional embeddings use sine and cosine functions of different frequencies to encode position information.

Mathematical Formulation: For a given position pos\text{pos}pos and dimension i:

$$PE_{(\text{pos},\ 2\text{i})} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(\text{pos},\ 2\text{i}+1)} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right)$$

Where d is the embedding dimension.

Strengths:

- No additional parameters to learn.
- Ensures smooth positional transitions due to the continuous nature of sine and cosine functions.
- Effective for sequences longer than those seen during training due to the extrapolative properties of sinusoids.

Weaknesses:

- Fixed and non-adaptive to the specific data or task.
- Might not capture complex positional relationships as effectively as learned embeddings.

Learned Positional Embeddings

Learned positional embeddings treat positions similarly to tokens, learning a unique embedding vector for each position during training.

Mathematical Formulation: For a maximum sequence length N and embedding dimension d:

$$P \in \mathbb{R}^{N \times d}$$

Where P is a matrix of learnable parameters.

Strengths:

- Adaptable to the specific dataset and task, potentially capturing more complex positional relationships.
- Simple to implement and integrate into existing models.

Weaknesses:

- Fixed maximum sequence length, which limits handling of longer sequences.
- Requires learning additional parameters, increasing the model complexity and training time.

Relative Positional Embeddings

Relative positional embeddings encode the relative distances between tokens rather than absolute positions. This approach is particularly useful in capturing local dependencies and patterns.

Mathematical Formulation: Incorporates relative distance information into the attention mechanism. For tokens at positions i and j:

$$a_{ij} = \frac{(Q_i K_j^T + Q_i R_{i-j}^T)}{\sqrt{d_k}}$$

Where R is a matrix of relative positional embeddings.

Strengths:

- Effective at capturing local dependencies and contextual information.
- More flexible for varying sequence lengths and permutations.

Weaknesses:

- Can be more complex to implement.
- May introduce additional computational overhead during the attention calculation.

Rotary Positional Embeddings (RoPE)

Rotary positional embeddings introduce a rotation operation to encode positional information directly into the attention mechanism.

Mathematical Formulation: Applies a rotational matrix to the key and query vectors in the self-attention mechanism.

$$q_i' = q_i \cos(\theta_i) + \text{Rot}(q_i) \sin(\theta_i)$$

$$k_i' = k_i \cos(\theta_i) + \text{Rot}(k_i) \sin(\theta_i)$$

Where θ is the positional angle, and Rot is a rotation function.

Strengths:

- Efficiently integrates positional information into the attention mechanism.
- Reduces positional embedding size while maintaining performance.
- Effective for longer sequences due to the periodic nature of rotations.

Weaknesses:

- More complex to implement compared to absolute positional embeddings.
- Requires careful tuning of rotational parameters for optimal performance.

Conclusion

Each type of positional embedding has its unique strengths and weaknesses:

- **Sinusoidal Positional Embeddings**: Parameter-free and extrapolative but non-adaptive.
- **Learned Positional Embeddings**: Adaptable and simple but with fixed length and increased complexity.
- **Relative Positional Embeddings**: Captures local dependencies effectively but can be computationally intensive.
- **Rotary Positional Embeddings (RoPE)**: Efficient and suitable for long sequences but complex to implement.

Positional embeddings are a vital component in the transformer architecture, enabling models to process sequential data effectively. The choice of positional embedding method can significantly impact model performance and efficiency, depending on the specific application and data characteristics.

Mathematics | Artificial Intelligence | Machine Learning | Deep Learning | Neural Networks